O algoritmih in podatkovnih strukturah

Tomaž Dobravec, Algoritmi in podatkovne strukture 2

O algoritmih in podatkovnih strukturah

- **Prepletenost algoritmov in podatkovnih struktur**
  - Podatkovna struktura – uporablja algoritme za učinkovito izvedbo.
  - Algoritem – uporablja podatkovno strukturo za obdelavo podatkov.
  - Ko govorimo o enih, imamo v mislih tudi druge (in obratno!)

- **Kakovost algoritmov (in podatkovnih struktur)**
  - Zanima nas dvoje: **pravilnost** in **učinkovitost**

  - **PRAVILNOST:**
    - skoraj vedno pravilen rezultat
    - skoraj pravilen rezultat
    - vedno popolnoma pravilen rezultat

  - **UČINKOVITOST**
    - količina porabljenih virov

    - algoritem, ki porabi manj virov, je bolj učinkovit

Časovna zahtevbnost

- Zakaj nas zanima čas izvajanja izbranega algoritma?

- Kdaj pride do izraza razlika v času izvajanja med 1ms in 2ms?

- V kakšni enoti merimo čas izvajanja algoritmov pri teoretični analizi? Zakaj?

Zanimajo nas **parametri**, ki opisujejo nalogo in **funkcija**, ki dane vrednosti parametrov preslika v število korakov.

Časovna zahtevnost - primeri

Približno ocenjevanje časovne zahtevnosti

- Natančne ocene pogosto ne poznamo.
- Zadovoljimo se z oceno zgornje meje
- Malo natančnejša analiza → velikostni red funkcije časovne zahtevnosti.

- Uporaba simbolov:

  - **Θ** ... velikostni red

  - **O** ... zgornja meja

  - ***Ω*** *... spodnja meja*

Kako veliko je veliko?

- Pomembna je predstava o tem, kaj pomenijo asimptotske ocene v konkretnih primerih!

  Primer: imam algoritem, s časovno zahtevnostjo $O(n^2)$ in podatke velikosti $n=10^7$.
  Koliko časa (sekund/minut/ur?) bo treba počakati, da algoritem konča?

  Tabela: Časi izvajanja pri $n=10,\ 20,\ 100,\ 1000,\ 10^6,\ 10^9$ za različne časovne zahtevnosti

- Zakaj je eksponentna časovna zahtevnost problematična?
  - Primer 1: zgodba o rižu
  - Primer 2: prepogibanje listov papirja

- Izziv: sprogramiraj preprost algoritem za množenje matrik.
  - Algoritem poženi na različno velikih vhodih.
  - Meri čas in preveri pravilnost ocene za časovno zahtevnost $T(n)=C*n^3$.
  - Kako velika je konstanta C na tvojem računalniku?

Kako veliko je veliko? - Tabela časovnih zahtevnosti v sekundah

Kako veliko je veliko? - Dva primera eksponentnega naraščanja

Dodane rešitve in dopolnitve za eaa263f7-76f0-4079-ad23-58eeea71b9a7.pdf

`eaa263f7-76f0-4079-ad23-58eeea71b9a7.pdf` - uvod, zahtevnost

Stran 3 - zakaj merimo čas teoretično

- Čas izvajanja nas zanima, ker želimo primerjati algoritme neodvisno od konkretnega računalnika.
- Razlika med 1 ms in 2 ms postane pomembna pri velikem številu ponovitev ali velikih vhodih.
- V teoriji časa ne merimo v sekundah, ampak v številu osnovnih korakov kot funkcijo velikosti vhoda `n`.
- Če je `T(n)=C*f(n)`, pri asimptotiki konstante in nižje člene ignoriramo.

Strani 6-8 - kako veliko je veliko

Za grobo oceno velja:

- `O(log n)` in `O(n)` sta običajno zelo dobra.
- `O(n log n)` je praktičen za velika urejanja.
- `O(n^2)` hitro postane problem pri milijonih elementov.
- `O(2^n)` in `O(n!)` sta uporabna samo za zelo majhne `n`.

Mini trik: če v nalogi piše 1 GHz, računaj približno `10^9` osnovnih operacij/s.